MDPI

Article

Benchmarking Local Large Language Models for Structured Knowledge Extraction in Conversational Commerce

Ratomir Karlović [1], Luka Sever [1], Marijela Miličević [1], Sandi Baressi Šegota [1], Vedran Mrzljak[2] and Ivan Lorencin [1]

[1] Faculty of Informatics, Juraj Dobrila University of Pula, Zagrebačka 30, 52100 Pula, Croatia; ratomir.karlovic@unipu.hr (R.K.), luka.sever@unipu.hr (L.S.), marijela.milicevic@unipu.hr (M.M.), sandi.baressi.segota@unipu.hr (S.B.Š.), ivan.lorencin@unipu.hr (I.L.)
[2] Faculty of Engineering Rijeka, University of Rijeka, Vukovarska 58, 51000 Rijeka, Croatia;
* Correspondence: vedran.mrzljak@riteh.uniri.hr;

Abstract

The integration of natural language understanding into conversational commerce requires robustly extracting structured intents from idiomatic text. While cloud-based models are highly capable, their latency, privacy risks, and operational costs necessitate efficient local alternatives. To address this, we empirically evaluate locally deployed, open-weights large language models (LLMs) on parsing multi-intent shopping cart commands into deterministic JSON schemas. We benchmark multiple models across minimal, extended, and few-shot prompting paradigms, utilizing a dual-axis framework to simultaneously measure extraction quality (operation F1-score, exact match, schema validity) and hardware performance (latency, GPU power consumption, VRAM usage). Our evaluation highlights a fundamental trade-off between schema compliance and computational overhead, demonstrating how advanced prompting techniques improve parsing accuracy while measurably impacting inference latency and power draw. Ultimately, this comprehensive profiling identifies optimal model configurations for resource-constrained environments, establishing a practical baseline for deploying cost-effective and energy-efficient LLMs in edge-based retail automation. Ultimately, this comprehensive profiling identifies optimal model configurations for resource-constrained environments: a 3–8B parameter model paired with few-shot prompting recovers over 95% of the quality of the largest evaluated model at roughly 40× lower energy per command, establishing a practical and quantified baseline for deploying cost-effective LLMs in edge-based retail automation.

Keywords: LLM; Prompt Engineering; Natural Language Understanding; JSON Parsing; Structured Output; Model Performance; Conversational Agents; Information Extraction;

1. Introduction

The rapid evolution of Large Language Models (LLMs) has fundamentally transformed the landscape of Natural Language Processing (NLP), enabling advanced capabilities in reasoning, content generation, and semantic understanding [1]. While early applications focused primarily on open-ended text generation, modern production environments increasingly require LLMs to function as reliable semantic parsers capable of mapping unstructured user intent into rigorous, machine-readable formats such as JSON [2,3]. This capability is particularly critical in domain-specific applications, such as conversational commerce controllers, where valid schema adherence is a prerequisite for

Received:
Revised:
Accepted:
Published:

Copyright: © 2026 by the authors. Submitted to *Journal Not Specified* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

executing downstream backend tasks like inventory management and automated checkout workflows.

A primary challenge in deploying these systems within real-world retail architectures is balancing semantic parsing fidelity against operational and infrastructural constraints. While centralized, cloud-hosted proprietary models deliver high structural compliance, they introduce significant barriers regarding data privacy, volatile network latency, and high operational costs at scale [4]. Consequently, deploying open-weights models locally on edge or dedicated infrastructure has emerged as a compelling alternative to guarantee data sovereignty and predictable processing thresholds [5]. However, local deployments introduce an immediate engineering trade-off: smaller models frequently suffer from systemic structural failures, syntax corruption, or semantic omissions when tasked with generating strict object structures without heavy optimization.

In our previous work, we addressed these constraints by conducting a targeted empirical evaluation comparing Parameter-Efficient Fine-Tuning (PEFT) techniques against varied prompting complexities [1]. That study utilized a controlled synthetic dataset of 12,000 isolated, single-intent shopping directives (e.g., "Toss in 14 razors") to parse specific entities into a simple three-attribute JSON schema. While those findings established foundational baselines showing that adapter-based methods or dense few-shot contexts could mitigate performance gaps across model scales, the evaluation was intentionally isolated from the linguistic noise and structural density characteristic of live human interactions.

To address these limitations, the current study significantly scales the structural and linguistic complexity of the semantic parsing task. Rather than processing isolated, single-intent inputs, we investigate the capacity of local open-weights models to unpack compound, multi-intent shopping cart commands compressed into a single utterance (e.g., "Toss in 14 razors in there, throw in 20 pies, load up with 19 coffees, and also nix a pair of bananas"). Parsing these compound structures requires an LLM to simultaneously determine disparate entity boundaries, map variable informal action synonyms (such as "load up" to an addition or "nix" to a removal), and resolve implicit numeric representations (such as "a pair" to an integer value of 2) within a single inference pass.

The primary aim of this research is to evaluate how distinct inference-time prompt engineering paradigms alter the parsing accuracy and hardware footprint of local models when processing these highly complex inputs. We implement a systematic benchmarking matrix crossing three prompting archetypes (Minimal, Extended, and Few-Shot) against a parameter-scale continuum of six local models: granite4:3b, llama3.2:3b, llama3.1:8b, mistral:7b, gpt-oss:20b, and gemma4:31b. Rather than analyzing extraction quality in isolation, we utilize a dual-axis framework that maps parsing metrics (Op-F1, EM rate, and schema validity) directly against physical hardware execution profiles (token latency, VRAM utilization, and GPU power consumption). This empirical setup uncovers the direct computational cost of prompt optimization, providing a data-driven baseline for selecting cost-effective and energy-efficient configurations for resource-constrained conversational agents.

2. Background and Related Work

The integration of Large Language Models (LLMs) into conversational agents has necessitated robust strategies for adapting general-purpose architectures to specialized, deterministic tasks. In complex e-commerce environments, agents must navigate noisy, heterogeneous user intents where implicit requirements frequently challenge standard generation capabilities. Tou et al. [6] established the ShoppingComp benchmark to evaluate autonomous agents across precise product retrieval and safety-critical decision-making, revealing a substantial human-AI capability gap where state-of-the-art models struggle with

implicit requirement inference when translating vague intents into technical specifications. To evaluate the multi-step reasoning required to overcome these challenges, Wang et al. [7] introduced ShoppingBench, utilizing trajectory distillation to train a compact Qwen3-4B agent capable of complex budgeting tasks. Their findings highlighted that while smaller models can match closed-source baselines, attribute mismatching remains a persistent bottleneck. To optimize agents for these multi-objective interactions, Cheng et al. [8] demonstrated that post-training Reinforcement Learning, specifically through HRM and DCPO, significantly improves objective product correctness and operational tool efficiency without relying on generic reasoning paths.

To operate reliably in production environments, these models must exhibit high resilience to input perturbations and domain-specific constraints. Zhu et al. [9] evaluated LLM robustness against adversarial prompts, finding that while contemporary models suffer significant performance drops from word-level attacks, few-shot prompting and instruction fine-tuning are critical strategies for preventing attention divergence. Expanding on adaptation strategies, Wang et al. [10] demonstrated that fine-tuning open-weights models like Llama-3-8B and Qwen-7B for text classification effectively eliminates hallucinated outputs, reducing uncertainty rates to near zero compared to variable few-shot setups. In highly specialized domains, Dinh et al. [11] showed that translating natural language intents into machine-executable JSON or YAML configurations relies heavily on prompting design, with few-shot CoT prompting drastically mitigating format errors. Similarly, in the clinical domain, Balasubramanian et al. [12] and Grothey et al. [13] demonstrated that advanced open-source models offer viable, privacy-preserving alternatives for structured feature extraction, though specialized strategies like CoV are required to stabilize outputs upon 4-bit quantization. Extending structured extraction to scientific literature, Dagdelen et al. [14] utilized LoRA and human-in-the-loop workflows to fine-tune Llama-2 for relation extraction, showing that LLMs automatically normalize complex chemical entities into structured JSON. Yang et al. [15] applied similar multimodal QLoRA tuning to the RRA-Logis framework for edge-computing, mapping noisy logistics documents directly to JSON schemas with 100% compliance via a confidence-gated decision mechanism. Evaluating such capabilities on a global scale, Ling et al. [16] introduced ResumeBench for multilingual parsing. Their results indicated that while enabling explicit JSON mode boosts syntactic fidelity, reasoning-distilled models surprisingly underperform compared to standard instruct counterparts, establishing models like Qwen2.5-14B as highly efficient deployment baselines.

2.1. Structured Output Generation and Benchmarking

To ensure reliable integration into transactional workflows, evaluating model proficiency in generating strictly constrained outputs has led to dedicated empirical frameworks. Yang et al. [17] introduced StructEval, a benchmark spanning diverse text-only and visual structures, demonstrating that generating valid structural outputs from natural language is significantly more difficult than simple format conversion. Addressing the fragility of standard "return valid JSON" prompting, Villota [18] argued that relying on probabilistic generation leads to high failure rates, advocating instead for constrained decoding at inference time to guarantee 100% schema adherence for text classification and retrieval tasks. Supporting this paradigm, Geng et al. [19] developed JSONSchemaBench to evaluate frameworks like Guidance and Outlines, revealing that optimized constrained decoding not only guarantees structural validity but can also accelerate the generation process by up to 50% and improve downstream reasoning tasks. To enforce this fidelity without massive parameter scaling, Shen et al. [20] introduced SLOT, utilizing a fine-tuned lightweight model as a dedicated post-processing layer, while Escarda-Fernandez et al. [21] demon-

strated that QLoRA fine-tuning effectively overcomes zero-shot limitations when mapping natural language to custom JSON APIs on resource-constrained IoT devices.

When constrained decoding is not implemented, the choice of prompt style heavily dictates extraction success. White et al. [22] evaluated diverse prompt formats across frontier models, identifying a persistent trade-off: hierarchical formats like JSON provide high accuracy but incur higher token costs and latency compared to compact formats like CSV. Shorten et al. [23] explored these zero-shot limitations through the StructuredRAG benchmark, finding that task complexity rapidly degrades formatting reliability, though OPRO can effectively bridge the performance gap for smaller models like Llama 3 8B. Pushing beyond static prompting, Lu et al. [24] proposed Schema Reinforcement Learning (SRL), utilizing a fine-grained validator and a "Thoughts of Structure" mechanism to significantly boost the schema compliance of Llama models beyond standard supervised fine-tuning. Taking an inverse approach, Shrimal et al. [25] introduced the PARSE framework, treating the JSON schema itself as an optimizable interface. By iteratively refining entity descriptions and validation rules, their system reduced extraction errors by up to 92% across multiple model scales, proving that optimizing the target structure mitigates inference latency and self-correction cycles.

In our previous research, we explored the "optimization dilemma" of structured parsing by comparing PEFT against advanced prompting strategies for local LLMs [1]. Utilizing a dataset of 12,000 synthetic, single-intent shopping commands mapped to a rigid JSON schema, we found that LoRA-based adapters consistently yielded superior accuracy with zero inference-time overhead. However, the study also established that dense few-shot prompting provides a highly viable, training-free baseline that can mitigate performance gaps in smaller models, such as Llama 3.2 3B. The critical trade-off identified was that while prompt engineering requires no upfront training costs, it inherently increases long-term inference latency and computational resource allocation due to the extended token context required for multi-shot exemplars.

Based on the cumulative evidence from these studies, a critical gap remains in quantifying the exact hardware and latency costs of pure inference-time prompting when applied to highly complex, compound-intent inputs. While previous works have examined isolated intents or heavily orchestrated fine-tuning pipelines, the goal of this current study is to examine how distinct prompting archetypes influence both structured output reliability and physical hardware consumption. Guided by prior literature, the following hypotheses are considered:

- H1: Advanced prompt engineering paradigms, specifically those utilizing complex Few-Shot exemplars, will significantly improve multi-intent extraction accuracy and schema validity compared to minimal prompts across all evaluated model scales.
- H2: The structural and semantic accuracy gains achieved through extended context prompting will incur a measurable, non-linear increase in inference latency, peak VRAM allocation, and GPU power consumption.
- H3: Semantic mapping failures (e.g., incorrect value extraction or hallucinated intents) will persist more frequently in ultra-compact edge models ($\leq$ 3B parameters) despite advanced prompting, reflecting inherent limitations in compound reasoning compared to larger open-weights architectures.
- H4: Mid-tier local models (e.g., 7B to 8B parameters) utilizing advanced prompting will demonstrate the optimal cost-benefit ratio, bridging the structural capabilities of larger models while maintaining operational footprints suitable for edge deployment.

This paper consists of six main sections. Section 1 introduces the research problem and operational motivation. Section 2 reviews related work on local large language models and structured data extraction. Section 3 describes the empirical methodology, detailing the

multi-intent dataset, prompt variants, evaluated model continuum, and hardware profiling metrics. Section 4 presents the experimental results across both parsing accuracy and hardware efficiency axes. Section 5 synthesizes the findings, discusses the limitations of the evaluation, and outlines pathways for future research. Finally, Section 6 summarizes the core conclusions of the study.

3. Methodology

We evaluate local large language models on a single structured-extraction task using a dual-axis protocol that scores every model on both extraction quality and computational cost. The task, dataset, and serving stack are held fixed; only the model and the prompting regime vary. No model is fine-tuned—all are used off-the-shelf through in-context prompting—so the study isolates the behaviour of pretrained open-weights models under realistic, low-effort local deployment. The remainder of this section defines the extraction task (Section 3.1), the evaluation corpus (Section 3.2), the model pool (Section 3.3), the three prompting regimes (Section 3.4), the inference and measurement protocol (Section 3.5), and the quality and cost metrics (Section 3.6). The prompts are reproduced verbatim in Appendix A.

3.1. Task Formulation

Given a single natural-language utterance addressed to a shopping assistant, a model must return the set of cart operations the utterance encodes. Each operation is a triple $(action, product, quantity)$ in which $action \in \{\texttt{add}, \texttt{remove}\}$, *product* is the product name as it appears in the utterance (with action verbs and numerals removed), and $quantity \in \mathbb{Z}_{\geq 1}$, defaulting to 1 when the user gives no number. The required output is a JSON array of such triples, for example `[{"action":"add","product":"pies","quantity":20}]`. The task is intentionally demanding along three dimensions: (i) one utterance bundles several operations that must be jointly segmented and recovered; (ii) intent is conveyed through open-ended synonyms ("toss in", "ring up", "nix", "take out") that must be collapsed onto the two canonical actions; and (iii) quantities are written as digits, number words, and idioms—"a pair" $\rightarrow$ 2, "half a dozen" $\rightarrow$ 6, "two dozen" $\rightarrow$ 24—that must be normalised to integers. A correct response therefore couples intent classification, verbatim span extraction, and quantity normalisation, emitted as schema-conformant JSON in a single forward pass.

3.2. Evaluation Corpus

We evaluate on a purpose-built corpus of 5000 shopping-cart commands, each paired with a gold list of operations. The corpus was generated with a frontier large language model (Anthropic Claude Opus 4.8 at maximum reasoning effort), prompted to emit colloquial, lexically varied utterances together with their exact structured parse; a randomly drawn 5% subset (250 commands) was then manually inspected to confirm that the gold operations matched the utterance. The corpus is used solely for evaluation: because no model is trained or tuned, there is no train/test split, and every command contributes to the reported metrics. It should be noted that using a frontier model to generate the evaluation corpus introduces a potential stylistic bias: the gold-label parses reflect the output conventions of the generating model, and evaluated models whose output style is closer to that model may receive a marginal advantage. This risk is mitigated by the deterministic nature of the extraction task—the gold operations are fully constrained by the utterance content—but cannot be entirely excluded.

Table 1 summarises the corpus. It spans 149 distinct retail products and 18,175 gold operations, a mean of 3.64 operations per command, with every command carrying between two and five. Additions outnumber removals (62.6% vs. 37.4%), and quantities are integers

in $[1, 25]$ with a mean of 11.0. Because each utterance carries several operations and mixes additions with removals, exact recovery demands that the model track multiple intents at once rather than predict a single dominant action.

Table 1. Composition of the evaluation corpus: distribution of commands by the number of gold operations they contain (5000 commands, 18,175 operations, mean 3.64 per command).

Operations per command	2	3	4	5	Total
Commands	427	1840	1864	869	5000
Share (%)	8.5	36.8	37.3	17.4	100.0

3.3. Models

We benchmark six open-weights, instruction-tuned models that span roughly an order of magnitude in scale (from 3B to 31B parameters) and five families—Meta Llama, Google Gemma, Mistral, IBM Granite, and OpenAI gpt-oss (Table 2). Every model was obtained from the Ollama library using its default tag and the default quantization published for that tag (predominantly 4-bit; gpt-oss in its native MXFP4 format); none was re-quantized or fine-tuned. This reproduces the configuration a practitioner obtains from a single `ollama pull`, which is precisely the low-effort, on-premises deployment scenario the paper targets.

Table 2. Evaluated models, all served through Ollama (v0.21.2-rc0) using the default tag and quantization. Parameter counts are the nominal sizes indicated by the model tag; "MoE" marks a MoE model.

Ollama tag	Developer / family	Params	Architecture
`gemma4:31b`	Google / Gemma	31B	Dense
`gpt-oss:20b`	OpenAI / gpt-oss	20B	MoE
`llama3.1:8b`	Meta / Llama 3.1	8B	Dense
`mistral:7b`	Mistral AI / Mistral	7B	Dense
`granite4:3b`	IBM / Granite 4.0	3B	Dense
`llama3.2:3b`	Meta / Llama 3.2	3B	Dense

3.4. Prompting Regimes

Each model is run under three system prompts of increasing specification while the user turn (the raw command) is held identical, so that any difference in output is attributable to the prompt alone. The minimal prompt states the task and the three output fields in a few lines. The extended prompt adds explicit instructions—chiefly that synonymous verbs must be mapped onto `add`/`remove` and that an unspecified quantity defaults to one—but gives no examples. The few-shot prompt instead pairs the schema with three worked input–output exemplars that demonstrate synonym mapping, idiomatic-quantity normalisation ("a pair" $\rightarrow$ 2, "a dozen" $\rightarrow$ 12), and the single-item default. Contrasting minimal with extended isolates the marginal value of instruction detail, whereas contrasting extended with few-shot isolates the marginal value of in-context demonstration. The full prompt texts appear in Appendix A.

3.5. Inference and Measurement Protocol

All models were served locally with Ollama (v0.21.2-rc0) on a single compute node providing two NVIDIA A100-SXM4 GPUs (40 GB each, 80 GB aggregate), a 32-core AMD EPYC 7763 CPU, and 120 GB of system memory; the runtime shards the larger models across both GPUs. Each command was issued once, using Ollama's default decoding parameters—no temperature, top-p, context-length, or random-seed overrides—so that the measurements reflect default out-of-the-box behaviour. Generation is consequently stochastic, and each of the $6 \times 3 = 18$ model–prompt configurations is evaluated in a single

pass over the 5000 commands (90,000 generations in total); we do not average over repeated runs and therefore treat run-to-run variance as a limitation of the design.

For every request we record the wall-clock latency around the generation call, the prompt- and output-token counts reported by the runtime, and the generation time. GPU telemetry is sampled from `nvidia-smi` immediately after each request: power draw and utilisation are averaged across the two GPUs, while memory usage is summed. From the aggregated samples we derive a first-order per-command energy estimate, $E \approx \bar{P}\,\bar{t}$, the product of mean (per-GPU) power $\bar{P}$ and mean latency $\bar{t}$; we report it as an indicative figure rather than a metered measurement.

3.6. Evaluation Metrics

Each configuration is scored on a quality axis and a cost axis, both computed directly from the stored responses.

Response parsing. Output is first reduced to operations by a lenient parser: we attempt a direct JSON parse and, on failure, extract the longest bracket-delimited substring (first `[` to last `]`, or first `{` to last `}`) and re-parse. This tolerates incidental wrapping such as prose preambles or Markdown code fences, so JSON validity credits any response from which a structured payload can be recovered; the schema and content metrics below then impose the stricter requirements.

Quality metrics. (i) JSON validity—the fraction of responses yielding a parseable payload. (ii) Schema validity—the fraction whose parsed operations are all well-typed objects carrying exactly the keys `action` ($\in \{$`add`, `remove`$\}$), `product` (string), and `quantity` (integer); an empty result is invalid. (iii) Operation-level micro-averaged F1 (Op-F1), the primary quality scalar: each operation is treated as the exact triple $(\mathit{action}, \mathit{product}, \mathit{quantity})$, and for each command we take the multiset overlap of predicted and gold operations, accumulating true positives, false positives, and false negatives over the entire corpus before computing a single micro-averaged precision, recall, and F1. Op-F1 is order-independent but requires all three fields to match exactly. (iv) Exact match (EM)—the fraction of commands whose predicted operation list equals the gold list exactly and in order, a strict whole-utterance criterion. (v) Field accuracies for action, product, and quantity—predicted and gold operations are aligned by position up to the longer length, and we report the fraction of aligned slots whose field agrees; being position-aligned, these conflate ordering and count errors with genuine field errors and are reported only as a diagnostic decomposition.

Cost metrics. Per configuration we report latency (mean, median, and 95th percentile, in seconds), throughput (commands per second over the run), mean GPU power (W, averaged over the two GPUs), per-command energy ($E \approx \bar{P}\,\bar{t}$, J), mean resident VRAM (GB, summed over both GPUs), and the mean prompt- and output-token counts. For the trade-off analysis (Section 4.3) quality is summarised by Op-F1 and cost by energy per command, the two scalars that anchor the Pareto comparison.

Generative-AI disclosure. In accordance with MDPI policy, we disclose that generative AI was used in a single part of this study—to synthesise the evaluation corpus (Section 3.2)—and nowhere else; model evaluation, measurement, analysis, and manuscript preparation used no generative-AI assistance beyond superficial text editing.

4. Results and Discussion

We evaluate six open-weights models under three prompting regimes—minimal, extended, and few-shot—yielding 18 model–prompt configurations, each applied to the full set of 5000 shopping-cart commands. These commands contain a mean of 3.64 atomic operations (18,175 gold operations in total), so a single utterance typically bundles several add/remove intents that must be jointly recovered. Following the dual-axis protocol

introduced in the Methodology, we report extraction quality (Table 3) and computational cost (Table 4) for every configuration, and then analyse the quality–cost trade-off that the prompting regime induces (Figure 6). Throughout, we use Op-F1 as the primary quality scalar and an estimated energy per command, $E \approx \bar{P}\,\bar{t}$ (mean GPU power $\times$ mean latency), as the primary cost scalar.

4.1. Extraction Quality and the Role of the Prompting Regime

Table 3. Extraction quality by model and prompting regime (Min, minimal; Ext, extended; FS, few-shot). JSON and Schema denote the JSON- and schema-validity rates; Op-F1 is the operation-level micro-averaged F1 score; EM is the ordered full-command EM rate. All values are computed over the 5000-command evaluation set. The best Op-F1 per model is shown in bold.

Model	Prompt	JSON	Schema	Op-F1	EM
Gemma4 31B	Min	1.000	1.000	0.951	0.833
	Ext	1.000	1.000	0.967	0.885
	FS	1.000	1.000	0.981	0.933
GPT-OSS 20B	Min	1.000	1.000	0.923	0.753
	Ext	0.999	0.999	0.944	0.819
	FS	1.000	1.000	0.975	0.915
Llama3.1 8B	Min	0.935	0.881	0.782	0.538
	Ext	0.998	0.976	0.801	0.568
	FS	0.997	0.995	0.950	0.841
Mistral 7B	Min	0.963	0.877	0.689	0.357
	Ext	0.982	0.865	0.504	0.281
	FS	0.993	0.976	0.942	0.804
Granite4 3B	Min	1.000	0.928	0.797	0.512
	Ext	1.000	0.931	0.790	0.516
	FS	1.000	0.997	0.937	0.807
Llama3.2 3B	Min	0.967	0.606	0.610	0.311
	Ext	0.970	0.637	0.526	0.229
	FS	0.890	0.822	0.813	0.566

Few-shot prompting yields the best Op-F1 for all six models, and the magnitude of the gain is inversely related to the model's zero-shot capability. Relative to the minimal prompt, in-context demonstrations raise Op-F1 by only $+0.031$ for Gemma4 31B and $+0.051$ for GPT-OSS 20B, but by $+0.140$ (Granite4 3B), $+0.168$ (Llama3.1 8B), $+0.203$ (Llama3.2 3B), and $+0.253$ (Mistral 7B) for the smaller models. The exemplars thus act as a capability equaliser: models that are mediocre zero-shot parsers become competitive once shown the target format, whereas the largest models—already near ceiling—have little headroom to gain. Figure 1 visualises this compression of the quality gap between small and large models.

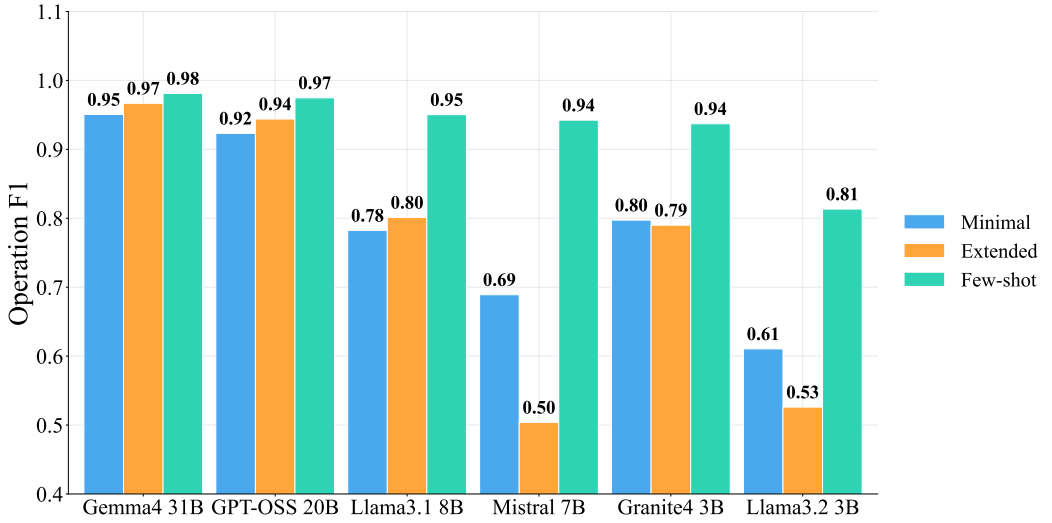

Figure 1. Op-F1 by model and prompting regime. Few-shot prompting (teal) yields the best accuracy for all six models and sharply compresses the gap between small and large models; the extended prompt (orange) is inconsistent, degrading Mistral 7B and the Llama3.2 3B small model relative to the minimal prompt (blue).

Decomposing quality into format adherence and content correctness clarifies the mechanism. Under the minimal prompt, several small models emit syntactically valid JSON that nonetheless violates the target schema: schema validity is only 0.928 (Granite4 3B), 0.881 (Llama3.1 8B), and 0.606 (Llama3.2 3B) despite JSON validity at or above 0.94. Few-shot demonstrations repair this format gap almost entirely—schema validity rises to 0.997, 0.995, and 0.822, respectively—indicating that the dominant failure mode of small models is structural (key naming, value typing, nesting) rather than semantic. Demonstrations specify the output contract by example rather than by description, which the smaller models follow more reliably than prose instructions. Figure 2 makes the format-versus-content split explicit: JSON validity sits at or near the ceiling for almost every configuration, so the small-model failures under weaker prompts are schema violations rather than malformed JSON (the corresponding schema-validity rates appear in Table 3).

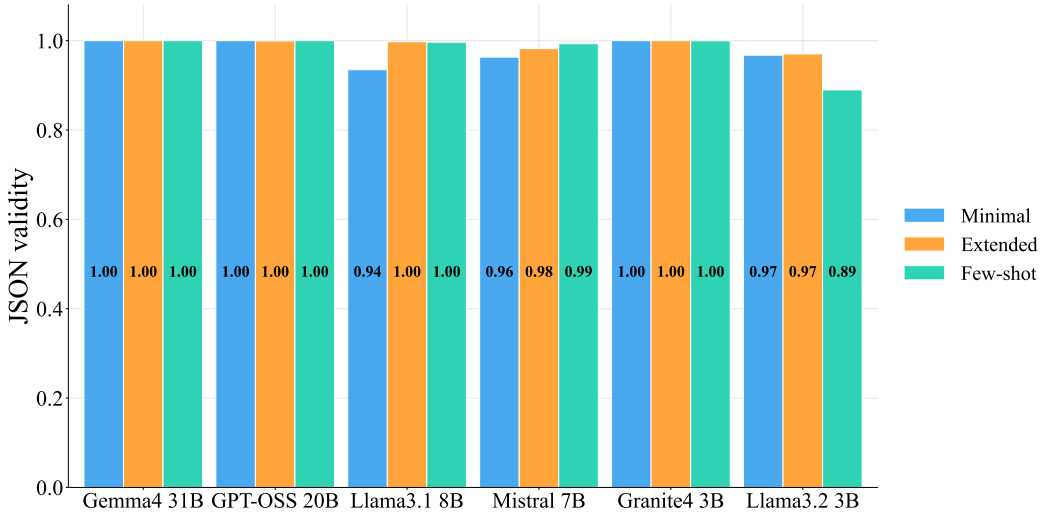

Figure 2. JSON validity by model and prompting regime (bars: minimal, extended, few-shot). Across almost all configurations the models emit parseable JSON at or near the ceiling; the residual small-model deficits under weaker prompts are predominantly schema violations rather than malformed JSON (cf. the schema-validity column of Table 3).

In contrast, the extended (instruction-rich) prompt is not a monotonic improvement and is frequently harmful. It helps only the most capable models—Gemma4 31B (+0.016), GPT-OSS 20B (+0.021), and Llama3.1 8B (+0.019)—while degrading Mistral 7B (−0.185) and Llama3.2 3B (−0.084), and leaving Granite4 3B essentially unchanged (−0.007). Manual inspection traces the Mistral regression to over-normalisation: the additional instructions induce the model to lemmatise product spans ("toothpastes" → "toothpaste", "quinoa bags" → "quinoa bag") and to misresolve idiomatic quantities ("a pair" → 1), collapsing product-field accuracy from 0.762 to 0.559 even though the emitted JSON remains well-formed. Longer instructions therefore introduce degrees of freedom that smaller models resolve in unintended, ostensibly "helpful" ways that diverge from the verbatim-span ground truth. This is an important negative result: scaling prompt complexity without grounding it in exemplars can reduce extraction fidelity.

4.2. Computational Cost and the Decode-Bound Regime

Table 4. Computational cost by model and prompting regime. Lat_{med}, Lat_{mean}, and Lat_{p95} are latency percentiles (s); Thr is throughput (commands/s); Pwr is mean GPU power (W); E is the estimated energy per command (J, $E \approx \bar{P}\,\bar{t}$); P-tok and E-tok are the mean prompt- and output-token counts. Lower is better for all columns except throughput.

Model	Prompt	Lat_{med}	Lat_{mean}	Lat_{p95}	Thr	Pwr	E (J)	P-tok	E-tok
Gemma4 31B	Min	13.03	16.33	37.24	0.06	156	2555	146	600
	Ext	13.25	16.27	36.62	0.06	156	2545	197	599
	FS	10.10	11.71	19.87	0.08	156	1820	436	426
GPT-OSS 20B	Min	2.46	2.54	3.90	0.36	124	315	196	345
	Ext	2.44	2.52	3.95	0.36	124	313	248	341
	FS	2.39	2.51	3.89	0.37	126	315	455	339
Llama3.1 8B	Min	0.72	0.71	0.98	1.06	141	101	138	79
	Ext	0.64	0.66	0.93	1.12	141	94	191	72
	FS	0.58	0.56	0.71	1.26	140	79	402	57
Mistral 7B	Min	0.61	0.59	0.84	1.24	146	87	148	88
	Ext	0.58	0.56	0.78	1.30	147	82	200	82
	FS	0.50	0.49	0.64	1.42	143	70	467	70
Granite4 3B	Min	0.65	0.61	0.82	1.21	119	73	136	98
	Ext	0.53	0.52	0.81	1.35	118	62	189	81
	FS	0.41	0.39	0.51	1.64	118	46	400	56
Llama3.2 3B	Min	0.51	0.53	0.76	1.34	123	65	148	90
	Ext	0.50	0.49	0.63	1.42	124	61	201	80
	FS	0.40	0.39	0.48	1.67	122	47	412	56

The cost profile is governed by decoding rather than prefill. Although few-shot prompts are three- to fourfold longer on the input side (prompt tokens rise from ∼140 to ∼400–470), they consistently reduce wall-clock latency because the demonstrations suppress verbose generation: mean output length falls (e.g., 98 → 56 eval tokens for Granite4 3B and 600 → 426 for Gemma4 31B), and decode dominates latency in autoregressive inference. Consequently, few-shot mean latency is 17.7–35.6% lower than minimal for five of the six models (Granite4 3B −35.6%, Gemma4 31B −28.3%, Llama3.2 3B −27.2%, Llama3.1 8B −21.2%, Mistral 7B −17.7%), with GPT-OSS 20B effectively flat (−1.4%). The conventional intuition that richer prompts cost more compute is thus inverted here: by constraining the output, exemplars make inference simultaneously more accurate and faster. Figures 3 and 4 contrast the prompt and output token counts that drive this effect.

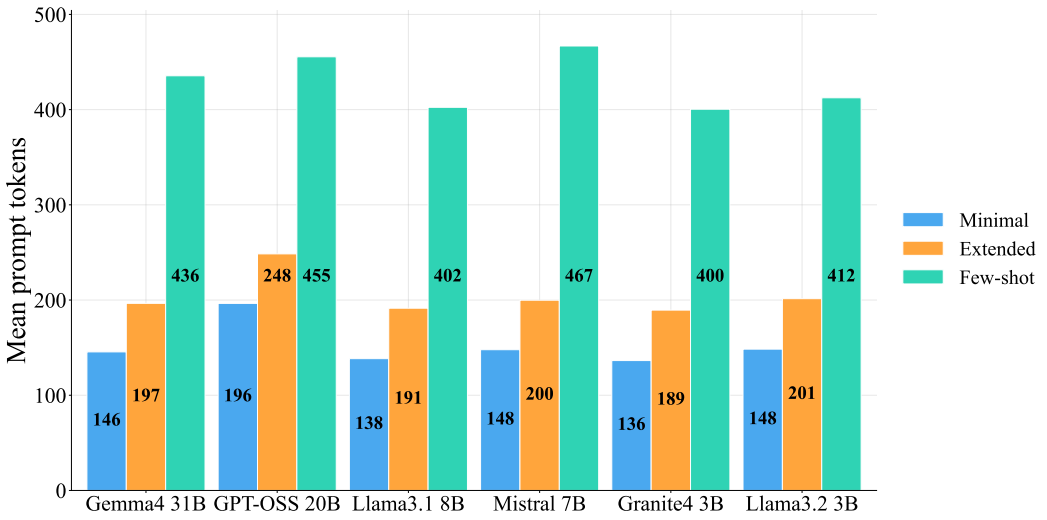

Figure 3. Mean prompt (input) tokens per command, by model and prompting regime. Few-shot prompts are markedly longer on the input side because each carries three worked exemplars, whereas the minimal and extended prompts are far shorter.

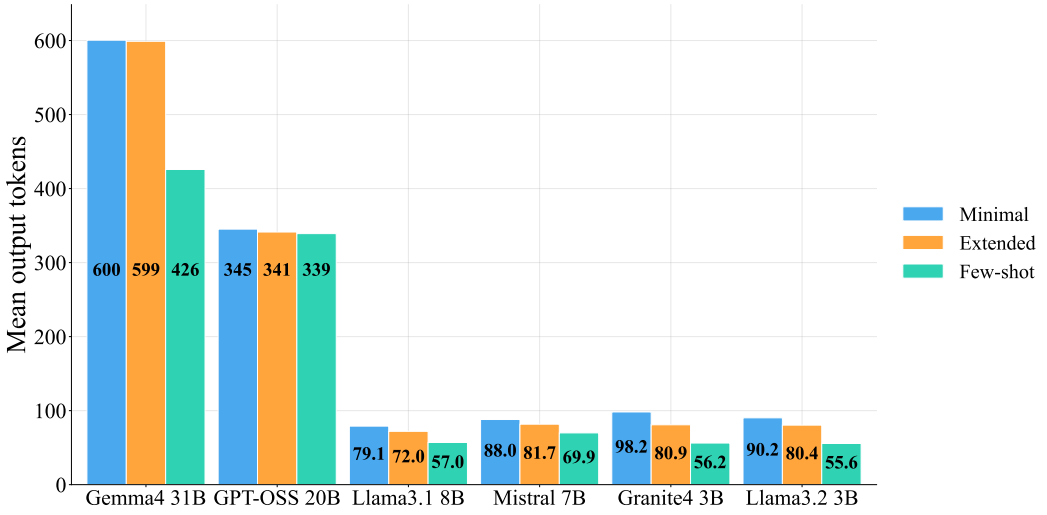

Figure 4. Mean output (decoded) tokens per command, by model and prompting regime. Despite the longer input, few-shot prompting yields shorter output: the demonstrations elicit concise, minified JSON and suppress verbose generation. Because decoding dominates latency, this reduction in output tokens is the mechanism behind the lower few-shot latency and energy reported in Table 4.

GPU power draw is essentially model-bound rather than prompt-bound—within each model it varies by only a few watts across regimes (e.g., ∼156 W for Gemma4 31B, ∼124 W for GPT-OSS 20B, ∼118 W for Granite4 3B)—so the estimated energy per command tracks latency (Figure 5). Few-shot prompting therefore reduces per-command energy by 19–36% for the capable models, with the smallest models reaching 46 J (Granite4 3B) and 47 J (Llama3.2 3B) per command versus 315 J (GPT-OSS 20B) and 1820 J (Gemma4 31B). Energy spans a factor of roughly 40 across the frontier—a far wider dynamic range than quality—which is the crux of the deployment trade-off analysed in Section 4.3.

We report mean VRAM occupancy in the underlying logs for completeness but caution against ranking models by it: the measured footprint does not track parameter count (the 20B GPT-OSS model occupies only ∼16 GB, less than the 3B Llama3.2 at ∼17 GB or the 8B Llama3.1 at ∼22 GB), which indicates that the figure reflects server-level allocator reservation and key–value-cache pre-allocation under the serving runtime rather than the

minimal model footprint. Absolute VRAM values should accordingly be read as an upper bound on residency for this deployment, not as an intrinsic model property.

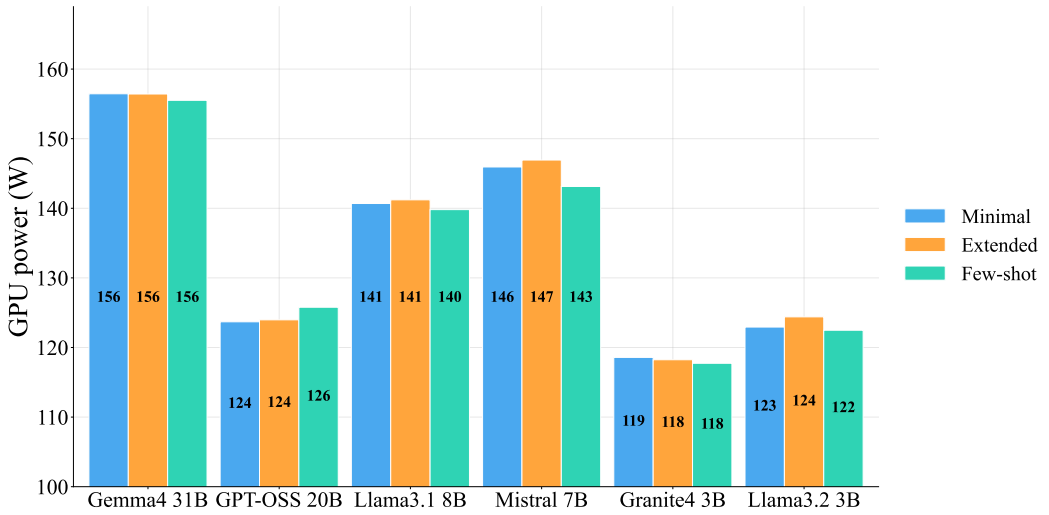

Figure 5. Mean GPU power draw (averaged across the two A100 GPUs) by model and prompting regime (bars: minimal, extended, few-shot). Power is essentially model-bound: within each model it is nearly invariant across prompting regimes, so per-command energy ($E \approx \bar{P}\,\bar{t}$) is governed by latency rather than by power. The remaining cost metrics—latency, throughput, energy, VRAM, and token counts—are reported in Table 4.

4.3. The Quality–Cost Trade-off

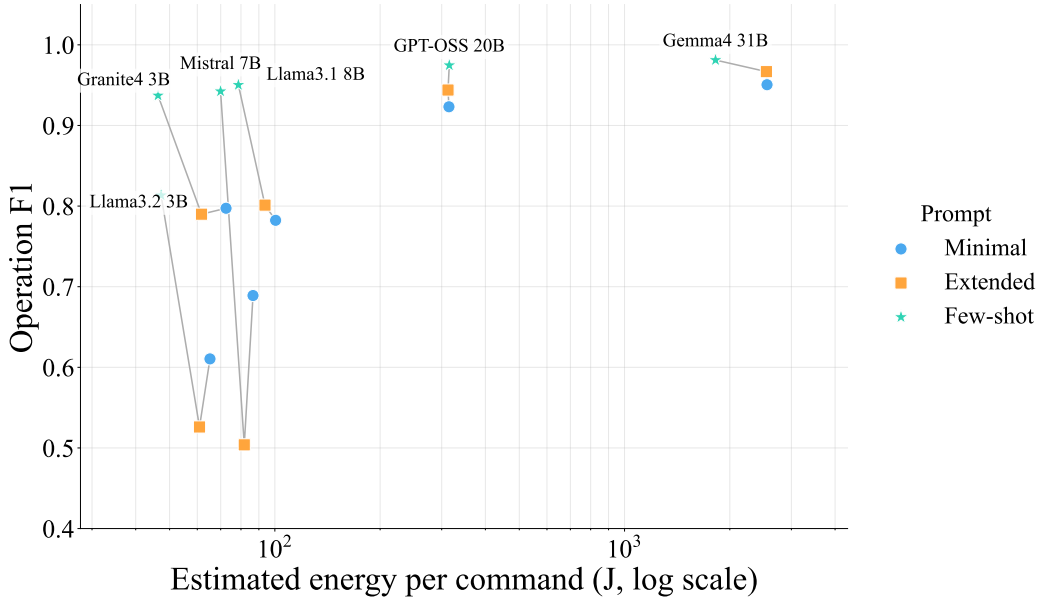

Figure 6. Quality–cost trade-off across all 18 configurations. Each point is one model–prompt configuration; thin grey lines connect the three regimes of a model. The horizontal axis is the estimated energy per command on a logarithmic scale. For every model the few-shot configuration (star) lies above and to the left of its minimal (circle) and extended (square) counterparts—simultaneously more accurate and more energy-efficient—so the operative trade-off is across model scale along the few-shot frontier rather than across prompts.

Figure 6 plots every configuration in the energy–quality plane and reveals the central finding of this study: for capable models, few-shot prompting is Pareto-improving rather than Pareto-trading—it moves each model up (higher Op-F1) and to the left (lower energy)

at once. The few-shot points dominate their own minimal and extended counterparts for all six models. The genuine trade-off is therefore not between prompts but across model scale: along the few-shot frontier, quality and cost are positively coupled, and the practitioner's decision reduces to selecting an operating point on that frontier. Three operating points are practically distinct:

- Maximum accuracy. Gemma4 31B (few-shot) attains the highest fidelity (Op-F1 0.981, EM 0.933) but at 1820 J and 11.7 s per command—appropriate only where accuracy is paramount and latency and energy are unconstrained, such as offline or server-side processing.
- Balanced. GPT-OSS 20B (few-shot) reaches Op-F1 0.975 at 315 J and 2.5 s, a 5.8× energy reduction for a 0.6-point Op-F1 loss relative to Gemma4 31B.
- Efficiency frontier. Granite4 3B (few-shot) achieves Op-F1 0.937—95.5% of Gemma4 31B's quality—at 46 J and 0.39 s, that is, roughly 40× less energy and 30× lower latency. Llama3.1 8B (0.950, 79 J) and Mistral 7B (0.942, 70 J) offer marginally higher accuracy at modestly higher cost.

Quality saturates rapidly with scale once few-shot prompting is applied: moving from a 3B to a 31B model improves Op-F1 by only ∼0.044 while increasing energy by a factor of roughly 40. For the resource-constrained, edge-oriented retail setting that motivates this work, a 3–8B model with few-shot prompting is therefore the rational default, reserving 20–31B models for accuracy-critical or server-side deployments. Crucially, this recommendation is contingent on the prompt: the same small models are not viable under minimal or extended prompting (Op-F1 0.50–0.80), so the deployment win arises from the prompt–model pairing rather than from the model alone. The same Pareto structure holds when mean latency replaces energy on the cost axis (Table 4), confirming that the ranking of configurations is robust to the choice of cost metric.

4.4. Error Analysis

Field-level accuracies localise the residual errors. Product-span extraction is the hardest field for every model that makes appreciable errors: under the minimal prompt the 3–8B models score only 0.70–0.87 on the product field, below both their action (0.76–0.93) and quantity (0.75–0.93) accuracies, reflecting sensitivity to surface form such as pluralisation; quantity resolution is additionally stressed by idiomatic numerals ("a pair", "half a dozen", "a dozen") that must be mapped to integers. For the near-ceiling large models the three fields all exceed 0.95 and their differences are negligible. The gap between Op-F1 and whole-command EM is informative: even Gemma4 31B (few-shot) scores 0.981 Op-F1 but 0.933 EM, because a command counts as correct only if all of its 3.64 operations on average are simultaneously correct, so per-operation errors compound multiplicatively at the command level. EM is consequently the more stringent and deployment-relevant metric for transactional settings, where a single misparsed item corrupts the entire cart update.

5. Limitations and Future Work

Several limitations of the present study should be acknowledged when interpreting the reported results and considering their generalisability.

5.1. Single-pass evaluation

Each of the 90,000 model–prompt–command combinations was generated exactly once without repeated runs or fixed random seeds, as Ollama's default decoding parameters were not overridden. The reported metrics therefore reflect a single stochastic sample, and run-to-run variance remains unquantified. Future work should average results over

multiple independent passes to obtain confidence intervals and assess result stability, particularly for configurations near performance boundaries.

5.2. Synthetic evaluation corpus

The 5,000-command evaluation set was generated by a frontier model (Anthropic Claude Opus 4.8) rather than collected from real user interactions. Although a 5% manual inspection confirmed gold-label correctness, the corpus may not fully capture the lexical diversity, grammatical irregularities, and implicit context of genuine conversational commerce sessions. Evaluation on corpora derived from live retail deployments is an important direction for future work.

5.3. Corpus generation bias

Because the evaluation corpus was generated by a single frontier model, the gold parses reflect one model's interpretation of idiomatic expressions and implicit quantities. Edge cases that the generating model resolves consistently but arbitrarily—for example, whether a few maps to 2 or 3—are embedded in the ground truth without an independent human reference. Future work should complement the synthetic corpus with a human-annotated subset to quantify the impact of this ambiguity on reported metrics.

5.4. Constrained decoding not evaluated

All results are obtained through pure prompt engineering without constrained decoding frameworks such as Outlines or Guidance, which can enforce schema validity at the token level and guarantee 100% structural compliance regardless of model size. Integrating constrained decoding into the evaluation matrix would allow a direct comparison between inference-time prompt optimisation and decoding-level schema enforcement, and is a natural extension of this work.

5.5. Single hardware platform

All experiments were conducted on a dedicated server node equipped with two NVIDIA A100-SXM4 GPUs. While this environment is representative of a data-centre or high-end on-premises deployment, it does not reflect the hardware constraints of true edge devices such as NVIDIA Jetson modules or Apple Silicon systems. Latency, power draw, and throughput figures will differ substantially on such platforms, and replicating the benchmark on constrained edge hardware is necessary to validate the efficiency claims made for the 3–8B model tier.

5.6. No fine-tuning baseline

The study deliberately isolates inference-time prompt engineering by evaluating all models off-the-shelf without any fine-tuning. While this reproduces the low-effort deployment scenario targeted by the paper, it does not establish how prompt-engineered configurations compare against adapter-based methods such as LoRA on the same compound multi-intent task.

Our prior work addressed this comparison for single-intent inputs; a direct extension to the multi-intent setting would complete the picture.

5.7. Schema complexity and domain scope

The extraction target is a fixed three-field schema covering a single retail domain with 149 product types. Real conversational commerce agents must handle richer schemas, nested structures, and diverse product taxonomies. Evaluating structured extraction across broader schema configurations and retail verticals would strengthen the generalisability of the reported findings.

6. Conclusions

We presented a dual-axis benchmark of six open-weights large language models, served entirely on local hardware, for the structured extraction of shopping-cart operations from idiomatic, multi-intent commands. Across 18 model–prompt configurations evaluated on 5000 commands, we jointly quantified extraction quality (Op-F1, EM, and schema validity) and the computational cost of inference (latency, throughput, GPU power, energy, and VRAM), so that accuracy and deployability are weighed on the same footing.

Returning to the hypotheses stated in the introduction, the results provide the following answers. H1 is confirmed: few-shot prompting significantly improves Op-F1 and schema validity across all six model scales, with gains inversely proportional to zero-shot competence. H2 is partially confirmed and partially inverted: advanced prompting does incur higher peak VRAM allocation through longer prompt contexts, but contrary to expectation it reduces rather than increases inference latency and energy, because the output suppression effect of demonstrations outweighs the prefill overhead. H3 is confirmed for minimal and extended prompts but not for few-shot: sub-3B models exhibit persistent structural failures under weaker prompts, yet with few-shot prompting Granite4 3B achieves Op-F1 0.937, which is competitive with models four to eight times larger. H4 is confirmed: the 7–8B tier under few-shot prompting offers a strong cost–quality compromise, though the 3B tier with few-shot prompting proves a viable alternative for the most constrained deployments.

Three findings stand out. First, in-context demonstration is the single most effective intervention: few-shot prompting produced the best Op-F1 for every model, and its benefit was inversely proportional to a model's zero-shot competence, compressing the quality gap between the smallest and largest models from wide to marginal. The dominant failure mode of the smaller models under weaker prompts was structural—schema-violating JSON rather than malformed text or incorrect content—and exemplars repaired it almost entirely, indicating that the output contract is conveyed far more reliably by example than by description. Second, longer instructions are not a free improvement: the instruction-rich extended prompt degraded several smaller models by inducing over-normalisation of product spans and idiomatic quantities, a negative result that cautions against scaling prompt complexity without grounding it in examples. Third, on this task richer prompting is not more expensive but cheaper: because decoding dominates latency and the demonstrations suppress verbose generation, few-shot prompting simultaneously raised accuracy and lowered per-command latency and energy, inverting the common assumption that better prompts cost more computation. GPU power, by contrast, was essentially model-bound, so energy per command tracked latency rather than prompt length.

Taken together, these results reframe the deployment decision. Because few-shot prompting is Pareto-improving rather than Pareto-trading, the operative choice is not which prompt to use—few-shot, always—but where to sit on the few-shot quality–cost frontier. Along that frontier quality saturates quickly while cost does not: moving from a 3B to a 31B model improves operation F1 by only $\sim$0.044 yet increases energy per command roughly fortyfold. For the resource-constrained, edge-oriented retail automation that motivates this work, a 3–8B model paired with few-shot prompting is therefore the rational default—Granite4 3B, for instance, recovers 95.5% of the 31B model's quality at $\sim 40\times$ less energy and $\sim 30\times$ lower latency—with larger models reserved for accuracy-critical or server-side processing. Crucially, this efficiency is a property of the prompt–model *pairing* and not of the model alone, since the same small models are not viable under minimal or extended prompting. Beyond the specific rankings, the study contributes a reusable, energy-aware evaluation protocol and an empirical baseline for deploying cost-effective local LLMs in conversational commerce; extending it to broader retail intents, multilingual

commands, and constrained-decoding methods that enforce schema validity directly are promising directions for future work.

Author Contributions: For research articles with several authors, a short paragraph specifying their individual contributions must be provided. The following statements should be used "Conceptualization, X.X. and Y.Y.; methodology, X.X.; software, X.X.; validation, X.X., Y.Y. and Z.Z.; formal analysis, X.X.; investigation, X.X.; resources, X.X.; data curation, X.X.; writing—original draft preparation, X.X.; writing—review and editing, X.X.; visualization, X.X.; supervision, X.X.; project administration, X.X.; funding acquisition, Y.Y. All authors have read and agreed to the published version of the manuscript.", please turn to the CRediT taxonomy for the term explanation. Authorship must be limited to those who have contributed substantially to the work reported.

Funding: This work was (partially) supported by the EC Digital Europe Programme EDIH Adria 2.0 (101256325); SPIN projects IP.1.1.03.0120, IP.1.1.03.0158 and IP.1.1.03.0039; and NextGenerationEU University grants: uniri-iz-25-6, uniri-iz-25-220, IIP_010144, IIP_010136

Data Availability Statement: The synthetic evaluation corpus, the generation-and-evaluation harness (`generate_fast.py`), and the plotting scripts used to produce the figures and tables in this paper are available from the authors and will be deposited in a public repository upon publication.

Acknowledgments: This research was performed using the Advanced computing service provided by University of Zagreb University Computing Centre - SRCE.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

LLMs	Large Language Models
NLP	Natural Language Processing
PEFT	Parameter-Efficient Fine-Tuning
LoRA	Low-Rank Adaptation
QLoRA	Quantized Low-Rank Adaptation
IA^3	Infused Adapter by Inhibiting and Amplifying Inner Activations
JSON	JavaScript Object Notation
YAML	Yet Another Markup Language
SFT	Supervised Fine-Tuning
F1	F1 Score
Op-F1	Operation-level F1 Score
EM	Exact Match
GPU	Graphics Processing Unit
VRAM	Video Random Access Memory
HRM	Hierarchical Reward Modeling
DCPO	Dynamic Contrastive Policy Optimization
CoT	Chain-of-Thought
CoV	Chain-of-Verification
MoE	Mixture-of-Experts
OPRO	Optimization by Prompting
SRL	Schema Reinforcement Learning
IoT	Internet of Things

Appendix A. Prompt Templates

The three system prompts are reproduced verbatim below. In every configuration the user turn contains only the raw command; the system prompt is the sole experimental manipulation.

Appendix A.1. Minimal Prompt

```
You are an assistant that analyzes user input related to a shopping cart. Your task is to extract cart
     operations from the user's message.
Return ONLY a JSON array where each item has:
- action (one of: 'add', 'remove')
- product (the product name exactly as the user mentions it)
- quantity (an integer; if not specified, assume 1)
Output format:
[{ "action": "add", "product": "...", "quantity": 1 }]
```

Appendix A.2. Extended Prompt

```
You are a shopping cart assistant. Your task is to extract cart operations from the user's message:
- 'action': classify the intent as either 'add' or 'remove', even if the user uses other words or
     phrases (e.g., 'put', 'insert', 'delete', 'take out', 'nix', 'ring up', 'pack in', 'skip' etc.).
- product: the exact product name as mentioned by the user
- quantity: an integer (default to 1 if not specified).
Return only a valid JSON array of objects, with no explanations, formatting, or extra text. The output
     must follow this format exactly:
[{ "action": "...", "product": "...", "quantity": ... }]
```

Appendix A.3. Few-Shot Prompt

```
You are a shopping-cart assistant whose only job is to parse the user's request and output a JSON array
      where every element has this exact schema:

{
  "action": "<add|remove>",
  "product": "<exact product name>",
  "quantity": <integer>
}

Rules:
1. "action" must be either "add" or "remove". Map any synonyms ("put in", "insert", "take out", "nix",
     "delete", etc.) to these two.
2. "product" is exactly what the customer wants, stripped of any action words or numbers.
3. "quantity" is an integer. If the user does not specify a number, default to 1.
4. Output ONLY the JSON array - no markdown, no explanations, no extra keys or text.

Examples:

User: Toss in 14 razors in there, throw in 20 pies, load up with 19 coffees, and also nix a pair of
     bananas
Output:
[
{"action":"add","product":"razors","quantity":14},
{"action":"add","product":"pies","quantity":20},
{"action":"add","product":"coffees","quantity":19},
{"action":"remove","product":"bananas","quantity":2}
]

User: Yo lose a dozen soaps, stick in a few potatoes in there, then take away a couple wraps
Output:
[
{"action":"remove","product":"soaps","quantity":12},
{"action":"add","product":"potatoes","quantity":3},
{"action":"remove","product":"wraps","quantity":2}
]

User: Add apples
Output:
[
{"action":"add","product":"apples","quantity":1}
]

Now parse the user's next message.
```

References

1. Karlović, R.; Sever, L.; Baressi Šegota, S.; Mrzljak, V.; Lorencin, I. Evaluating the Impact of Adapter-Based Fine-Tuning on Structured Parsing Performance in Large Language Models. *Machine Learning and Knowledge Extraction* **2026**, *8*. https://doi.org/10.3390/make8050138.
2. Daiber, J.; Maricato, V.; Sinha, A.; Rabinovich, A. DispatchQA: A Benchmark for Small Function Calling Language Models in E-Commerce Applications **2025**. pp. 2221–2233.
3. Tenckhoff, S.; Koddenbrock, M.; Rodner, E. Llmstructbench: Benchmarking large language model structured data extraction. *arXiv preprint arXiv:2602.14743* **2026**.
4. Ahmad, A.; Kowol, P.; Hillmann, S.; Möller, S. Multi-intent recognition in dialogue understanding: a comparison between smaller open-source LLMs. *arXiv preprint arXiv:2509.10010* **2025**.
5. Satija, S.; Bhatt, A.; Jani, P.; Rawal, D. fastWorkflow: Closing the Performance Gap Between Small and Frontier Language Models for Conversational Agents. In Proceedings of the Proceedings of the ACM Conference on AI and Agentic Systems, 2026, pp. 161–180.
6. Tou, H.; Zeng, Y.; Li, Y.; Ma, C.; Li, M.; Li, M.; Yuan, W.; Zhang, H.; Jia, K. ShoppingComp: Are LLMs Really Ready for Your Shopping Cart? *arXiv preprint arXiv:2511.22978* **2025**.
7. Wang, J.; Xiao, K.; Sun, Q.; Zhao, H.; Luo, T.; Zhang, J.D.; Zeng, X. Shoppingbench: A real-world intent-grounded shopping benchmark for llm-based agents. In Proceedings of the Proceedings of the AAAI Conference on Artificial Intelligence, 2026, Vol. 40, pp. 33521–33529.
8. Cheng, Y.; Mao, K.; Li, T.; Tan, J.; Wen, J.R.; Dou, Z. ChatShopBuddy: Towards Reliable Conversational Shopping Agents via Reinforcement Learning. *arXiv preprint arXiv:2603.06065* **2026**.
9. Zhu, K.; Wang, J.; Zhou, J.; Wang, Z.; Chen, H.; Wang, Y.; Yang, L.; Ye, W.; Zhang, Y.; Gong, N.; et al. Promptrobust: Towards evaluating the robustness of large language models on adversarial prompts. In Proceedings of the Proceedings of the 1st ACM workshop on large AI systems and models with privacy and safety analysis, 2023, pp. 57–68.
10. Wang, Z.; Pang, Y.; Lin, Y.; Zhu, X. Adaptable and reliable text classification using large language models. In Proceedings of the 2024 IEEE International Conference on Data Mining Workshops (ICDMW). IEEE, 2024, pp. 67–74.
11. Dinh, L.; Cherrared, S.; Huang, X.; Guillemin, F. Towards End-to-End Network Intent Management with Large Language Models. *arXiv e-prints* **2025**, pp. arXiv–2504.
12. Balasubramanian, J.B.; Adams, D.; Roxanis, I.; de Gonzalez, A.B.; Coulson, P.; Almeida, J.S.; García-Closas, M. Leveraging large language models for structured information extraction from pathology reports. *Journal of Pathology Informatics* **2025**, p. 100521.
13. Grothey, B.; Odenkirchen, J.; Brkic, A.; Schömig-Markiefka, B.; Quaas, A.; Büttner, R.; Tolkach, Y. Comprehensive testing of large language models for extraction of structured data in pathology. *Communications Medicine* **2025**, *5*, 96.
14. Dagdelen, J.; Dunn, A.; Lee, S.; Walker, N.; Rosen, A.S.; Ceder, G.; Persson, K.A.; Jain, A. Structured information extraction from scientific text with large language models. *Nature communications* **2024**, *15*, 1418.
15. Yang, L.; Li, D.; Zheng, S.; Kong, L.; Li, M.; Kong, F.; Wang, W. Research on Structured Extraction and Material Matching of Logistics Documents Based on Lightweight Large Language Models. *Applied Sciences* **2026**, *16*, 5641.
16. Ling, Z.; Zhang, H.; Cui, J.; Wu, Z.; Sun, X.; Li, G.; He, X. Beyond Human Labels: A Multi-Linguistic Auto-Generated Benchmark for Evaluating Large Language Models on Resume Parsing. In Proceedings of the Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, 2025, pp. 31907–31933.
17. Yang, J.; Jiang, D.; He, L.; Siu, S.; Zhang, Y.; Liao, D.; Li, Z.; Zeng, H.; Jia, Y.; Wang, H.; et al. StructEval: Benchmarking LLMs' Capabilities to Generate Structural Outputs. *arXiv preprint arXiv:2505.20139* **2025**.
18. Villota, J. Structured Data with LLMs Done Right: A Practical Guide to Classification, Retrieval and Generation with LLMs. *Retrieval and Generation with LLMs (October 21, 2025)* **2025**.
19. Geng, S.; Cooper, H.; Moskal, M.; Jenkins, S.; Berman, J.; Ranchin, N.; West, R.; Horvitz, E.; Nori, H. JSONSchemaBench: A Rigorous Benchmark of Structured Outputs for Language Models, 2025, [2501.10868].
20. Shen, Z.; Wang, D.Y.B.; Mishra, S.S.; Xu, Z.; Teng, Y.; Ding, H. SLOT: Structuring the Output of Large Language Models. In Proceedings of the Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track, 2025, pp. 472–491.
21. Escarda-Fernández, M.; Botana, I.L.R.; Barro-Tojeiro, S.; Padrón-Cousillas, L.; Gonzalez-Vázquez, S.; Carreiro-Alonso, A.; Gómez-Area, P. LLMs on the Fly: Text-to-JSON for Custom API Calling. In Proceedings of the SEPLN (Projects and Demonstrations), 2024, pp. 130–135.
22. White, J.; Elnashar, A.; Schmidt, D. Prompt Engineering for Structured Data A Comparative Evaluation of Styles and LLM Performance **2025**.
23. Shorten, C.; Pierse, C.; Smith, T.B.; Cardenas, E.; Sharma, A.; Trengrove, J.; van Luijt, B. Structuredrag: Json response formatting with large language models. *arXiv preprint arXiv:2408.11061* **2024**.

24. Lu, Y.; Li, H.; Cong, X.; Zhang, Z.; Wu, Y.; Lin, Y.; Liu, Z.; Liu, F.; Sun, M. Learning to generate structured output with schema reinforcement learning. In Proceedings of the Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2025, pp. 4905–4918.
25. Shrimal, A.; Jain, A.; Chowdhury, S.; Yenigalla, P. PARSE: LLM driven schema optimization for reliable entity extraction. In Proceedings of the Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track, 2025, pp. 2749–2763.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.